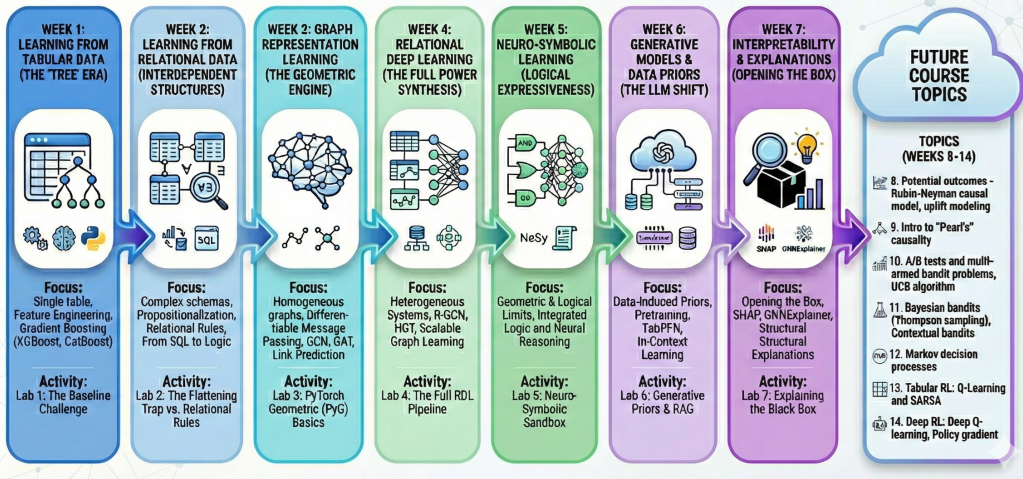


LEARNING FROM STRUCTURED DATA: A 7-WEEK JOURNEY

Narrative Arc: Increasing Structure → Increasing Inductive Bias → Increasing Reasoning Power



Week 1: Learning from Tabular Data (The "Tree" Era)

From The "Dark Matter" of AI to State-of-the-Art Gradient Boosting

BECM36MLM: Machine Learning Methods

Gustav Šír, PhD

The "Dark Matter" of Artificial Intelligence

- **Prevalence:** Tabular data powers $\sim 90\%$ of critical industry systems.
 - Fraud Detection, Healthcare, Supply Chain, Pricing.
- **Visibility:** Constitutes $< 10\%$ of current top-tier AI research (NeurIPS/ICML).
- **The Paradox:**
 - **Generative AI (Images/Text):** High Hype, Creative ROI.
 - **Tabular AI (Databases):** "Unsexy," High Financial ROI.

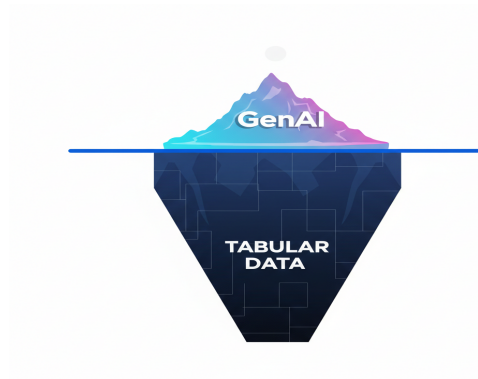


Figure: The AI Iceberg

The Fundamental Dichotomy

Feature	Perceptual Data (Images, Audio, Text)	Structured Data (Tabular, Relational)
Basic Unit	Pixel, Waveform, Token	Column / Field
Correlation	Spatial/Sequential: $x_{i,j} \approx x_{i+1,j}$	Semantic Indep.: "Age" $\perp$ "Zip"
Invariance	High (Rotation, Shift)	Low/None (Axis-Aligned)
Dims	High ($d \approx 10^3 - 10^6$)	Low to Medium ($d \approx 10^1 - 10^3$)
Priors	Strong (Physics/Grammar)	Weak (Domain Specific)

The "Correlation" Trap

- **Image:** A pixel is defined by its neighbors.
- **Table:** Column j (Income) may be statistically independent of Column $j + 1$ (User ID).

The Manifold Hypothesis

Definition

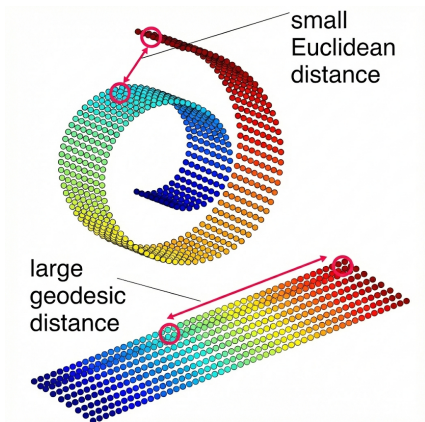
High-dimensional data ($x \in \mathbb{R}^d$) actually lies on a low-dimensional manifold ($\mathcal{M} \subset \mathbb{R}^d$) embedded within that space.

Deep Learning's Superpower:

Learning the coordinate transformation to unfold this manifold.

Unrolling the "Swiss Roll":

- Data points x_i exist in 3D.
- They lie on a 2D sheet rolled into a spiral.
- Euclidean distance is misleading.



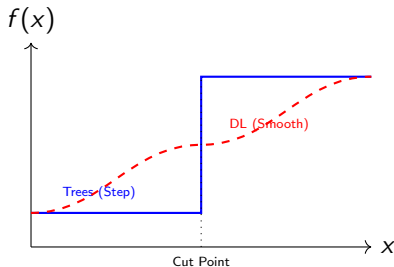
Tabular Data: The "Jagged" Reality

Finding: Neural Networks are biased toward overly smooth solutions.

The Problem: Tabular boundaries are non-smooth and axis-aligned.

$$f(x) = \mathbb{I}(x_{\text{Age}} > 25) \wedge \mathbb{I}(x_{\text{Inc}} > 50k)$$

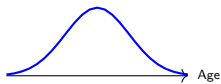
- This is a step function (discontinuous).
- High-frequency components are dominant.
- DL models "oversmooth" the boundary.



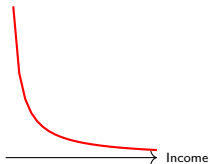
The Challenge of Heterogeneity

Data matrix $X \in \mathbb{R}^{N \times F}$ contains mixed distributions.

- **Dense Numerical:** Continuous, Gaussian-ish.
Ex: Temp, Age.
- **Sparse Categorical:** Discrete, no ordinality.
Ex: UserID, ZipCode (High Cardinality).
- **Heavy-Tailed:** Power laws.
Ex: Monetary amounts, Followers.



Normal



Heavy Tail



Sparse/High Card

Implication: DL struggles with unscaled outliers. Trees are invariant to monotonic transformations.

Structural Violations: I.I.D. and Missingness

1. The I.I.D. Assumption (Independent and Identically Distributed)

Assumption: $P(x_i, y_i) \perp P(x_j, y_j)$

Reality: Rows are often correlated via Time or Groups.

- **Time:** Fraud(Today) depends on Fraud(Yesterday).
- **Group:** Multiple rows for the same UserID.
- **Risk:** Random K-Fold CV leads to **Data Leakage**.

2. Informative Missingness

- **MCAR (Missing Completely at Random):** Noise.
- **MNAR (Missing Not at Random):** The fact that it is missing is a signal.
- *Example:* Income = NaN $\implies$ Status = Unemployed.
- **Model Implication:** Imputing with "Mean" destroys signal. Trees learn a "default direction" for NaNs.

Formal Notation for the Course

- **Dataset:** $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$
- **Input Vector:** $x_i = [x_{i,1}, x_{i,2}, \dots, x_{i,F}] \in \mathbb{R}^F$
 - Mixed types (Continuous $\mathbb{R}$, Categorical $\mathbb{Z}$, Binary $\{0, 1\}$).
- **Target:** $y_i \in \mathbb{Y}$
 - Regression: $\mathbb{Y} = \mathbb{R}$ — Binary Classification: $\mathbb{Y} = \{0, 1\}$
- **Optimization:**

$$\hat{\theta} = \arg \min_{\theta} \sum_{i=1}^N \mathcal{L}(y_i, f_{\theta}(x_i)) + \Omega(\theta)$$

- $\mathcal{L}$: Loss Function (e.g., MSE, LogLoss).
- Ω : Regularization term (Crucial for Trees).

Goal for Week 1: Finding the function form $f(x)$ that best handles the "jagged" nature of $\mathcal{D}$.

Week 1 Roadmap: The Ascent to SOTA

- ① **The Inductive Bias War:** Why Trees beat Neural Nets (Rotation Invariance).
- ② **Feature Engineering:** Target Encoding & Discretization (The fuel).
- ③ **Decision Trees:** The atomic unit. Recursive partitioning.
- ④ **Random Forests:** Variance reduction via Bagging.
- ⑤ **Gradient Boosting (GBM):** Bias reduction via Boosting (learning from residuals).
- ⑥ **XGBoost:** The Systems Engineering masterpiece.
 - Sparsity awareness, Approximate splits.

Output: By the end of this week, you will understand the mathematical and structural reasons why XGBoost/LightGBM are the default starting point.

The Challenger: Multilayer Perceptrons (MLP)

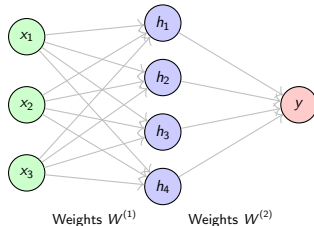
The Architecture: A standard Feed-Forward Network.

- **Input Layer:** $x \in \mathbb{R}^F$
- **Hidden Layers:** $h^{(k)} = \sigma(W^{(k)}h^{(k-1)} + b^{(k)})$
- **Output:** $y = \text{Softmax}(W^{(out)}h^{(L)})$

The Promise: Universal Approximation Theorem.

The Reality on Tabular Data

- Requires extensive hyperparameter tuning (LR, Dropout, Weight Decay).
- Training is slower than Gradient Boosted Trees (GBT).
- **Performance:** Frequently underperforms GBT on medium-sized tabular datasets.



Theory: Inductive Bias

Definition

The set of assumptions a learning algorithm makes to predict outputs for unseen inputs.

No Free Lunch Theorem: No model is universally superior. Performance depends on how well the bias aligns with the data generation process.

Comparing Biases:

- **Convolutional NN (CNN):**
 - Assumes **Translation Invariance** (cat top-left = cat bottom-right) and **Locality**.
- **Recurrent NN (RNN):**
 - Assumes **Sequential** dependencies.
- **Standard MLP:**
 - Assumes **Rotational Invariance** and **Smoothness**.
- **Tree Ensembles:**
 - Assumes **Axis-Aligned** decision boundaries.

The Nature of Tabular Information

Semantic Distinctness:

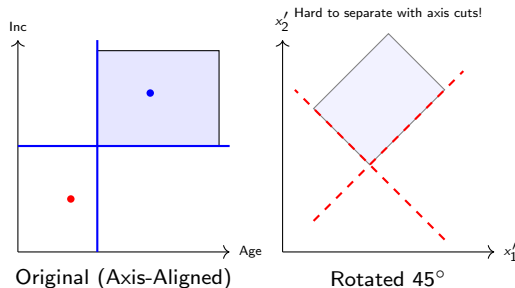
- Feature x_1 : "Age" (Years)
- Feature x_2 : "Income" (\$)

The Decision Boundary: Usually orthogonal to axes.

- Rule: IF Age > 25 AND Income > 50k
- Forms a **Hyper-rectangle**.
- Splits aligned with basis vectors e_1, e_2 .

Sensitivity to Rotation:

- Rotate by $45^\circ \rightarrow 0.7 \cdot \text{Age} + 0.7 \cdot \text{Inc} > C$.
- Destroys semantic simplicity.



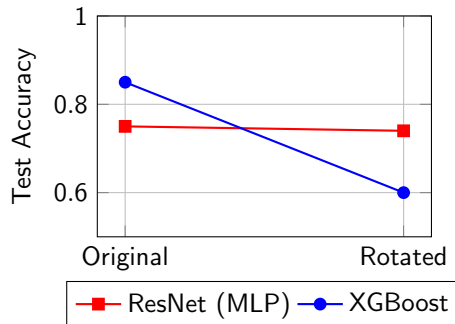
Empirical Proof: The Rotation Test

Finding: Data is Non-Invariant by Rotation.

Source: Grinsztajn et al. (Finding 3)

The Experiment:

- 1 Take standard tabular datasets.
- 2 Train Models (ResNet vs. XGBoost).
- 3 Rotate features X by orthogonal matrix R :
 $X_{rot} = XR$.
- 4 Retrain and measure Test Accuracy.



- ResNet: Stable (Invariant).
- XGBoost: Drops significantly.

Trees outperform on Original data $\implies$ Data is axis-aligned.

Neural Networks & The Smoothness Bias

Lazy Training & Spectral Bias

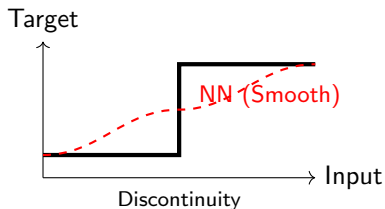
Neural Networks trained with SGD tend to learn low-frequency functions first.

The Tabular Mismatch:

- Tabular functions are often **Non-Smooth** (Step functions, Decision cliffs).
- *Example:* Tax brackets. A jump occurs exactly at income = \$100,000.

Comparison:

- **NN Behavior:** Fits a smooth sigmoid to the step. **Oversmoothing** → Errors in transition.
- **Tree Behavior:** Fits a step function natively. Matches the data generator.



The "Regularization Myth"

The Myth

"If MLPs overfit, just add Dropout or Weight Decay (L_2)."

The Reality:

- Standard regularization forces the model toward **smoothness** (small Lipschitz constant).
- We just established that tabular data is often **non-smooth**.

The Consequence:

- 1 **Heavy regularization** prevents the MLP from learning the necessary jagged boundaries.
- 2 **Low regularization** leads to overfitting on the noise.

Conclusion: The regularization needed for tabular data is **structural** (sparsity/discreteness), not just magnitude-based (L_2).

Robustness to Uninformative Features

Scenario: Dataset has 100 features, but only 10 are predictive. 90 are noise.

MLP Behavior

- Initializes weights for all 100 connections.
- Gradients from 90 noise features propagate into hidden layers.
- Must learn to set 90 weights to zero (Hard with L_2).

Tree Behavior

- Greedy splitting searches for best single feature.
- If feature is noise → Low Information Gain.
- Tree simply **never selects** the noise features.
- **Implicit Feature Selection.**

Summary: The Inductive Bias Scorecard

Property	Tabular Nature	Neural Net Bias	Tree Bias
Boundary	Jagged, Discontinuous	Smooth, Continuous	Step-wise
Orientation	Axis-Aligned	Rotational Invariance	Axis-Aligned
Features	Heterogeneous	Homogeneous	Handles Mixed
Noise	High Feature Noise	Vulnerable	Robust

Conclusion:

- Trees win on **Inductive Bias** for Tabular Data.
- Neural Networks win on **Representation Learning** (Images/Text).

Next Module: Since we are sticking with Trees, how do we prepare the data? → **Feature Engineering**.

Categorical Features: The Cardinality Problem

Low Cardinality: $\text{Color} \in \{\text{Red}, \text{Blue}, \text{Green}\}$

- **Solution:** One-Hot Encoding (OHE).
- Creates 3 binary columns. Sparse, manageable.

High Cardinality: $\text{ZipCode} \in \{10001, \dots, 99999\}$ (40k+ levels)

- **OHE Failure:** Creates 40,000 sparse columns.
- **Curse of Dimensionality:** Tree splits become inefficient; model overfits to rare categories.

The Alternative: Label Encoding?

- Map $10001 \rightarrow 1$, $99999 \rightarrow 40000$.
- **Problem:** Imposes an arbitrary ordinal relationship ($\text{Zip } 99999 > \text{Zip } 10001$). Trees struggle to "unlearn" this order.

Target Mean Encoding (Bin Counting)

Concept: Replace category x_i with the expected value of target y .

$$\tilde{x}_i = P(y = 1 | x_i = c) \approx \frac{\sum_{j \in \text{Train}} \mathbb{I}(x_j = c) \cdot y_j}{\sum_{j \in \text{Train}} \mathbb{I}(x_j = c)}$$

Intuition:

- If Zip=10001 has 5% fraud rate, replace "10001" with **0.05**.
- Projects high-dim categorical space $\rightarrow$ 1D Risk space.

Benefit: Trees split on Risk ($Risk > 0.04$) instead of 40k branches.

Raw Data

City	Target
NY	1
SF	0
NY	1
SF	0
CHI	0

Group By
& Mean $\rightarrow$

Mapping

City	Mean
NY	1.0
SF	0.0
CHI	0.0

New Feature:
NY $\rightarrow$ 1.0
SF $\rightarrow$ 0.0

The Trap: Data Leakage

Scenario

- 1 We have **1 row** for Zip=90210. The target is $y = 1$ (Fraud).
- 2 We calculate the mean: $1/1 = 1.0$.
- 3 We replace 90210 with **1.0**.

The Model's View:

- "Oh, whenever I see the feature value 1.0, the target is Fraud!"
- The model **memorizes the label** y_i because it is embedded in the feature x_i .

The Result:

- **Test Time:** A new transaction comes from 90210. The global mean might actually be 0.05, but the model learned a hard rule for the specific training value.
- **Massive Overfitting.**

Robust Target Encoding

1. Leave-One-Out (LOO) Encoding

When calculating the mean for row i , exclude row i from the calculation.

$$\tilde{x}_i = \frac{(\sum_j y_j) - y_i}{(\sum_j 1) - 1}$$

Result: The feature for row i depends only on **other people's** behavior.

2. Smoothing (Regularization)

If a category has only 2 samples, the mean is noisy. We shrink toward Global Mean.

$$\tilde{x}_i = \lambda(\text{CatMean}) + (1 - \lambda)(\text{GlobalMean})$$

- $\lambda = \frac{n}{n+m}$ where n is count, m is smoothing parameter.
- *Effect:* Rare categories $\rightarrow$ Global Average (Neutral prior).

Discretization (Binning)

Definition: Converting continuous variables into discrete buckets.

Why?

- **Non-Linearity:** Allows linear models to learn non-linear effects (e.g., Age < 18 vs > 65).
- **Outliers:** Cap outliers into the last bucket (e.g., Income $> 1\text{M}$).

Techniques:

- **Fixed-Width:** Equal range (0-10, 10-20...). Bad for skewed data.
- **Quantile Binning:** Equal frequency (0-25th, 25-50th...). Adapts to distribution.

Tree Implication

While Trees can learn splits on continuous data, **pre-binning** (e.g., into 256 integers) drastically speeds up training (Histogram-based methods in XGBoost/LightGBM).

Missing Values: Signal, Not Noise

Types of Missingness:

- **MCAR (Missing Completely At Random):** Data entry error. (Safe to drop/impute).
- **MNAR (Missing Not At Random):** The absence of data correlates with target.
- *Example:* In loan data, Debt_to_Income is NaN for people with no debt history.

NaN $\neq$ Mean. NaN $\neq$ 0. NaN is a specific state.

Engineering Strategy:

- **Separate Column:** Create Is_Missing binary flag.
- **Tree Handling (XGBoost):** Learns a "Default Direction".
 - At every node, it learns: "If value is Missing, go Left."
 - Groups NaNs with the cluster (High Risk/Low Risk) they statistically resemble.

Feature Engineering Checklist

① Categoricals:

- Avoid One-Hot for high cardinality.
- Use **Target Encoding** with Smoothing and LOO to prevent leakage.

② Numericals:

- Consider **Quantile Binning** to handle skew and outliers.
- Essential for Neural Networks, optional but fast for Trees.

③ Missing Data:

- Treat NaN as a valid category.
- Let the model learn the "Default Direction".

Next Up: We have the data prepared. Now we build the first model.

→ **Module 4: Decision Trees (The Recursive Logic)**

Module 4: Decision Trees

Key Topics:

- The Logical View: Disjunction of Conjunctions (DNF).
- The Geometric View: Recursive Hyper-rectangular Partitioning.
- The Algorithm: CART (Classification and Regression Trees).
- The Math: Impurity Measures (Entropy & Gini).

The Logic: Disjunction of Conjunctions (DNF)

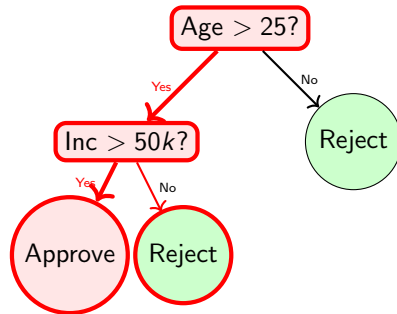
Structure: A Tree is a collection of paths from Root to Leaf.

The Path (Conjunction): To reach Leaf 1, you must satisfy:

$$(Age > 25) \wedge (Inc > 50k) \wedge (Debt = F)$$

This is an AND chain.

The Tree (Disjunction): The model predicts "High Risk" if (Path A is True) $\vee$ (Path B is True). *This is an OR of ANDs.*



The Geometry: Slicing the Hypercube

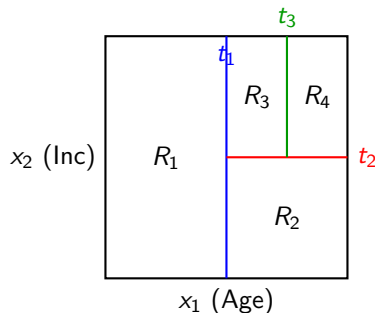
Input Space: Feature vector $x \in \mathbb{R}^F$.

The Mechanism:

- 1 Select dimension j (e.g., "Income").
- 2 Select cut point s (e.g., 50,000).
- 3 Slice space into R_1 ($x_j \leq s$) and R_2 ($x_j > s$).
- 4 Repeat recursively.

The Prediction: Constant value c_m per region R_m .

$$f(x) = \sum_{m=1}^M c_m \mathbb{I}(x \in R_m)$$



The CART Algorithm (Greedy Search)

The Problem

Finding the optimal tree is **NP-Complete**. We cannot check every possible combination of splits.

The Solution: Greedy Approach. At each step, find the single best split that maximizes immediate purity. Do not look ahead.

The Loop (for a node):

- 1 Iterate through every feature $j = 1 \dots F$.
- 2 Iterate through every possible threshold s for feature j .
- 3 Evaluate the cost of the split (j, s) .
- 4 Select the (j^*, s^*) with the lowest cost.
- 5 Split data and recurse.

Splitting Criteria: Regression

Goal: Group similar y values together.

Cost Function: Squared Error (Variance).

The Objective: Minimize the weighted variance of the children regions R_1, R_2 .

$$\min_{j,s} \left[\sum_{x_i \in R_1(j,s)} (y_i - \bar{y}_{R_1})^2 + \sum_{x_i \in R_2(j,s)} (y_i - \bar{y}_{R_2})^2 \right]$$

Where $\bar{y}_{R_k}$ is the mean target in region k .

Intuition: We want the child nodes to be "tighter" clusters of values than the parent node.

Splitting Criteria: Classification

Goal: Create "Pure" nodes (containing only one class).

Let p_{mk} be the proportion of class k observations in node m .

Metrics of Impurity:

- **Misclassification Error:** $1 - \max_k(p_{mk})$. (Not differentiable).
- **Gini Index (CART Default):**

$$H(Q_m) = \sum_k p_{mk}(1 - p_{mk})$$

- **Cross-Entropy (Information Theory):**

$$H(Q_m) = - \sum_k p_{mk} \log(p_{mk})$$

Information Gain:

$$\text{Gain} = H(\text{Parent}) - \left(\frac{N_L}{N} H(\text{Left}) + \frac{N_R}{N} H(\text{Right}) \right)$$

Handling Continuous & Categorical Features

1. Continuous Features ($Age \in \mathbb{R}$)

- Sort unique values: $v_1, v_2, \dots, v_n$.
- Consider split points: $\frac{v_i + v_{i+1}}{2}$.
- **Cost:** Sorting takes $O(N \log N)$. This is the bottleneck.

2. Categorical Features ($City \in \{NY, LA, SF\}$)

- **One-Hot/Binary:** Is $City == NY$? (Creates simple splits).
- **Set Splitting:** $City \in \{NY, SF\}$ vs $\{LA\}$.
- **Problem:** 2^{k-1} partitions. Exponential explosion!
- **Solution:** Target Encoding (Module 3) treats categories as numbers.

The Limit of Trees: Overfitting

The Phenomenon: A tree can continue splitting until every leaf contains exactly 1 sample.

- Training Error = 0.
- Test Error = High (Memorized noise).

Structural Regularization (Constraints):

- **Max Depth:** Limit layers (e.g., depth=6).
- **Min Samples per Leaf:** Stop splitting if node < 20 samples.

The Paradox

- A single "stump" (depth 1) has **High Bias** (Underfitting).
- A deep tree (depth 20) has **High Variance** (Overfitting).

Next Week: How do we get Low Bias AND Low Variance? → **Ensembles (Module 5).**

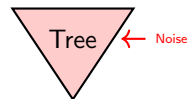
The Instability of a Single Tree

The Problem with Decision Trees:

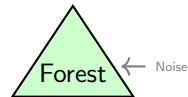
- **High Variance:** Small changes in training data → Large changes in tree structure.
- **Reason:** Hierarchical splits are conditional.
 - If Root split changes (e.g., Age → Income), the entire subtree changes.

Analogy: The "Physical Balance"

- **Single Tree:** Balancing on one foot. Highly responsive, but unstable. A gust of wind (noise) knocks you over.
- **Forest:** Standing on solid ground (crowd). Individual perturbations cancel out.



Unstable



Stable

Bagging: Bootstrap Aggregating

The Idea: Train many unstable models ($T_1, \dots, T_B$) on slightly different views of the data and average their predictions.

1. Bootstrap Sampling

- Given dataset $\mathcal{D}$ of size N .
- Create B datasets $\mathcal{D}_1, \dots, \mathcal{D}_B$ by sampling N items from $\mathcal{D}$ **with replacement**.
- *Result:* Each $\mathcal{D}_b$ contains $\sim 63.2\%$ of unique original samples; the rest are duplicates.

2. Aggregation

- **Regression:** $\hat{y} = \frac{1}{B} \sum_{b=1}^B T_b(x)$
- **Classification:** Majority Vote.

Why Averaging Works (The Math)

Let $T_1(x), \dots, T_B(x)$ be identically distributed predictions with variance σ^2 and pairwise correlation ρ .

The Variance of the Average

$$\text{Var} \left(\frac{1}{B} \sum_{b=1}^B T_b(x) \right) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2$$

- **Component 1:** $\rho \sigma^2$ (The asymptotic floor).
- **Component 2:** $\frac{1-\rho}{B} \sigma^2$ (Vanishes as $B \rightarrow \infty$).

The Critical Insight:

- Bagging reduces variance by increasing B .
- **BUT:** If trees are highly correlated (high ρ), the first term $\rho \sigma^2$ dominates.
- *Problem:* If one feature (e.g., "Income") is very strong, all bagged trees split on it first. ρ remains high.

Random Forest: Bagging + Feature Subsampling

The Fix: Force trees to be different by restricting their vision.

Feature Subsampling (The Tweak)

At **every split** in the tree (not just the root):

- 1 Randomly select a subset of m features from the total p features.
- 2 Find the best split **only among these m** .

Standard Heuristics:

- Classification: $m = \sqrt{p}$
- Regression: $m = p/3$

Effect on Variance:

- Strong predictors are occasionally "hidden".
- Gives weaker features a chance to form upper nodes.
- **Result:** ρ (Correlation) decreases significantly $\rightarrow$ Lower Total Variance.

Algorithm: Random Forest

1. Loop $b = 1$ to B (number of trees):

- ① **Bootstrap:** Draw sample $\mathcal{D}_b$ of size N from training data.
- ② **Grow Tree T_b on $\mathcal{D}_b$:**
 - *At each node:* Randomly select m features from p . Pick best split. Split node.
 - **Crucial:** Do not prune. Grow deep (low bias).

2. Output:

- Regression: $\frac{1}{B} \sum T_b(x)$
- Classification: Majority Vote of $\{T_b(x)\}_1^B$.

Free Validation: Out-of-Bag (OOB) Error

The Concept

In each bootstrap sample $\mathcal{D}_b$, approx. 36.8% of samples are left out (e^{-1}). These are the **Out-of-Bag (OOB)** samples for tree T_b .

The Mechanism:

- 1 For each observation x_i , identify all trees T_b where x_i was OOB (not in training).
- 2 Calculate prediction using *only* those trees.

Result:

- An unbiased estimate of test error computed during training.
- No need for a separate validation set (saves data!).
- OOB Error $\approx$ Cross-Validation Error.

Problem: We cannot visualize 500 trees (Black Box).

- Sum of Info Gain by feature j across all trees.
- *Bias*: Favors high-cardinality features.

- Record Baseline Accuracy (OOB).
- Randomly shuffle** column j .
- Measure Drop in Accuracy.
- "If I destroy info in J , how much does model suffer?"



Summary & The Bridge to Boosting

Feature	Assessment
Accuracy	Excellent baseline ("Hard to break")
Preprocessing	Minimal (Handles scaling, missingness)
Parallelism	High (Trees are independent)
Interpretability	Moderate (Feature Imp., but no rules)
Latency	Slow (Must evaluate B trees)

The Limitation

- Random Forests focus on **Variance Reduction**.
- They **do not reduce Bias**.
- If underlying trees are too shallow (underfit), averaging won't help.

Next Module: How do we reduce Bias? → **Gradient Boosting.**

Module 6: Gradient Boosting Machines (GBM)

The Core Shift:

- **Bagging (Random Forest):** Independent models, parallel, averaged to reduce **Variance**.
- **Boosting:** Sequential models, iterative, summed to reduce **Bias**.

The Mathematical Identity

Gradient Boosting $\equiv$ Gradient Descent performed in **Function Space**

Intuition: Additive Modeling (The Golfer)

Goal: Predict target y (The Hole).

- ❶ **Model 1 (The Drive):** $F_0(x)$. A naive guess (e.g., mean).
 - *Residual:* $y - F_0(x)$ (Distance to hole).
- ❷ **Model 2 (Approach):** $h_1(x)$.
 - **Task:** Predict the residual $y - F_0(x)$.
 - **Update:** $F_1(x) = F_0(x) + h_1(x)$.
- ❸ **Model 3 (The Putt):** $h_2(x)$.
 - **Task:** Predict remaining error $y - F_1(x)$.
 - **Update:** $F_2(x) = F_1(x) + h_2(x)$.

Final Model: Sum of all shots.

$$F(x) = F_0 + \sum_{t=1}^T \eta \cdot h_t(x)$$



Figure: Boosting as Vector Addition

Forward Stagewise Additive Modeling

Hypothesis Space: We seek a function $F(x)$ composed of a sum of weak learners $h(x; \theta)$ (e.g., small trees).

$$F_M(x) = \sum_{m=1}^M \alpha_m h(x; \theta_m)$$

The Optimization Problem:

$$\min_{\{\alpha_m, \theta_m\}_1^M} \sum_{i=1}^N \mathcal{L}(y_i, F_M(x_i))$$

The Greedy Solution (Stage-wise): Suppose we have $F_{m-1}(x)$. We want to add one new function h_m to reduce the loss.

$$(\alpha_m, \theta_m) = \arg \min_{\alpha, \theta} \sum_{i=1}^N \mathcal{L}(y_i, F_{m-1}(x_i) + \alpha h(x_i; \theta))$$

Difficulty: Solving for optimal θ (tree structure) directly is hard for arbitrary loss functions $\mathcal{L}$.

The "Aha!" Moment: Gradient Descent

Standard Gradient Descent

Parameter Space

- **Goal:** Find parameters $\mathbf{w}$ to min $L(\mathbf{w})$.
- **State:** Vector $\mathbf{w}_t \in \mathbb{R}^d$.
- **Gradient:** $g_t = \nabla_{\mathbf{w}} L(\mathbf{w})$.
- **Update:** $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta g_t$.

Gradient Boosting

Function Space

- **Goal:** Find function values $\mathbf{F}$ to min $L(\mathbf{F})$.
- **State:** Vector $\mathbf{F}_t = [F(x_1), \dots, F(x_N)]$.
- **Gradient:** $g_{im} = \left[\frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$.
- **Update:**
 $F_m(x) = F_{m-1}(x) - \eta \cdot \text{TreeFit}(g_m)$.

Key Insight: The "Residual" is simply the **Negative Gradient** of the loss function with respect to the prediction. We train the tree $h_m(x)$ to predict this gradient.

Case 1: Squared Error (Regression)

Loss Function:

$$\mathcal{L}(y, F) = \frac{1}{2}(y - F)^2$$

The Gradient:

$$\frac{\partial \mathcal{L}}{\partial F} = \frac{\partial}{\partial F} \frac{1}{2}(y - F)^2 = -(y - F)$$

The Negative Gradient:

$$-\frac{\partial \mathcal{L}}{\partial F} = y - F$$

Conclusion: For Squared Error, the negative gradient is exactly the standard residual.

- This is why intuitively we say boosting "fits residuals".
- But this is a special case!

Case 2: Log-Loss (Binary Classification)

Loss Function:

$$\mathcal{L}(y, p) = -y \log(p) - (1 - y) \log(1 - p)$$

where $p = \sigma(F) = \frac{1}{1+e^{-F}}$ (Sigmoid of raw score).

The Gradient (Chain Rule):

$$\frac{\partial \mathcal{L}}{\partial F} = \frac{\partial \mathcal{L}}{\partial p} \cdot \frac{\partial p}{\partial F} \implies -\frac{\partial \mathcal{L}}{\partial F} = y - p$$

The Algorithm:

- 1 Convert raw predictions to probability: $p_i = \sigma(F_{m-1}(x_i))$.
- 2 Calculate **Pseudo-Residual**: $r_i = y_i - p_i$ (e.g., $1.0 - 0.7 = 0.3$).
- 3 Fit regression tree to r_i .

Algorithm: Gradient Boosting Machine (Friedman)

① **Initialize** $F_0(x) = \arg \min_{\gamma} \sum \mathcal{L}(y_i, \gamma)$ (e.g., mean of y).

② **For** $m = 1$ **to** M :

① Compute **Pseudo-Residuals**:

$$r_{im} = - \left[\frac{\partial \mathcal{L}(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}}$$

② **Fit Tree**: Train regression tree $h_m(x)$ to targets r_{im} .

③ **Compute Leaf Weights**: Optimal step γ_{jm} for each leaf R_{jm} (Line Search).

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} \mathcal{L}(y_i, F_{m-1}(x_i) + \gamma)$$

④ **Update**: $F_m(x) = F_{m-1}(x) + \eta \sum \gamma_{jm} \mathbb{I}(x \in R_{jm})$.

③ **Output** $F_M(x)$.

Regularization: Shrinkage

The Learning Rate (η)

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

- The parameter: η (learning rate), typically 0.01 to 0.3.

Why?

- Gradient Descent is unstable if steps are too large (overshooting).
- **Shrinkage:** By multiplying the tree output by a small factor (e.g., 0.1), we slow down the learning.
- We require more trees (higher M), but the final model generalizes much better.

Trade-off:

- Low $\eta \rightarrow$ Needs high M (Computationally expensive, better accuracy).
- High $\eta \rightarrow$ Needs low M (Fast, risk of overfitting).

Stochastic Gradient Boosting

Inspiration: Bagging (Random Forests) showed that randomness reduces variance.

Implementation in GBM:

- At each iteration m , do not use the full dataset N .
- **Row Subsampling:** Sample a fraction f (e.g., 0.5) of rows without replacement. Train tree h_m on this subsample.
- **Column Subsampling:** Sample a subset of features for this tree.

Benefits:

- **Speed:** Fitting a tree on 50% data is faster.
- **Accuracy:** Prevents the model from memorizing outliers (variance reduction).
- *Result:* Modern GBMs (XGBoost, LightGBM) are hybrids of Boosting and Bagging.

Module 6 Summary

- 1 Boosting fits an additive model sequentially.
- 2 Residuals are just **negative gradients** of the Squared Error loss.
- 3 GBM generalizes this to any differentiable loss (LogLoss, Huber, etc.) by training trees on gradients.
- 4 Regularization (Shrinkage + Subsampling) is critical to prevent overfitting.

Next Up: Module 7

- We have the theory. How do we make it fast enough for millions of rows?
- **XGBoost**: The Systems Engineering masterpiece.
 - Second-order gradients (Hessian).
 - Sparsity-aware splitting.
 - Block structure computing.

Module 7: XGBoost & Modern Implementations

The "Big Three" Gradient Boosting Frameworks:

- ① **XGBoost (2016)**: The robust standard. Introduced 2nd-order optimization and sparsity handling.
- ② **LightGBM (Microsoft)**: Optimized for massive scale. Leaf-wise growth & Gradient-based sampling (GOSS).
- ③ **CatBoost (Yandex)**: Specialized for categorical data. Solves target leakage via Ordered Boosting.

Gradient Descent (GBM) $\xrightarrow{\text{Newton's Method}}$ XGBoost

Mathematical Upgrade: The Second Order Approximation

Standard GBM: Approximates Loss using first derivative (Gradient g_i).

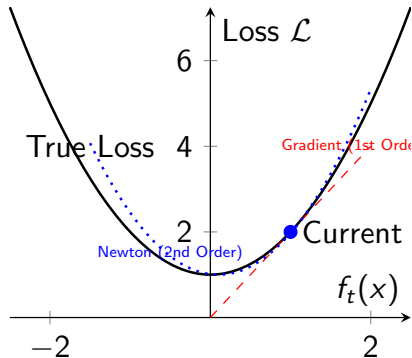
Analogy: Linear approximation (Tangent).

XGBoost: Approximates Loss using second derivative (Hessian h_i).

Analogy: Quadratic approximation (Parabola).

The Objective Function (at step t)

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^N \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$



Regularization: Penalizing Complexity

The Problem: Greedy boosting overfits.

The Solution: Bake regularization into the objective $\Omega(f)$.

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2$$

- γT : Penalty on the number of leaves (T).
- $\lambda \|w\|^2$: L2 penalty on leaf weights (w).

The Optimal Leaf Weight

By minimizing the quadratic objective, we get a closed-form solution:

$$w_j^* = - \frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$$

Intuition: If curvature $\sum h_i$ is small (flat loss, low confidence), λ dominates the denominator $\implies w_j^* \rightarrow 0$. *Conservative by default.*

System Optimization 1: Weighted Quantile Sketch

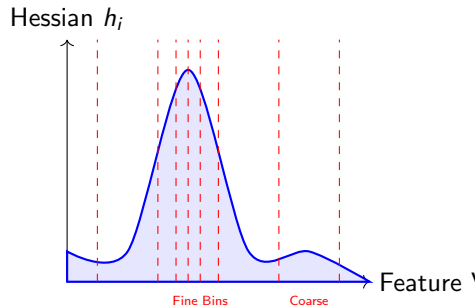
The Bottleneck: Exact Greedy Splitting requires sorting ($O(N \log N)$) at every node.

The Solution: Histogram-based splitting.

- Group feature values into B bins.
- Check splits only at bin boundaries.

The Innovation: Weighted Quantiles.

- Data points weighted by Hessian h_i .
- **High Curvature (Large h_i):** More bins (Fine resolution).
- **Low Curvature (Small h_i):** Fewer bins (Coarse resolution).



System Optimization 2: Sparsity Awareness

Challenge: Real-world data is sparse (Missing/Zero). Standard imputation adds noise.

XGBoost Algorithm: Learn a Default Direction

- 1 Identify **non-missing** values for feature j .
- 2 Enumerate splits only on non-missing values.
- 3 Calculate Gain sending all missing to **Left**.
- 4 Calculate Gain sending all missing to **Right**.
- 5 Pick the direction with max Gain.

Efficiency:

- Computation scales with number of non-zeros ($\|X\|_0$).
- $\sim 50\times$ speedup on very sparse data (e.g., One-Hot encoded).

LightGBM: GOSS & Leaf-wise Growth

Innovation 1: GOSS (Gradient-based One-Side Sampling)

- **Premise:** Points with small gradients g_i are well-trained.
- **Action:** Keep all instances with large gradients. Randomly sample small gradients.
- **Result:** Focuses training on "hard" examples.

Innovation 2: Leaf-wise Tree Growth

- **Level-wise (XGBoost):** Splits all nodes at depth d . (Balanced).
- **Leaf-wise (LightGBM):** Splits the single leaf with max loss reduction. (Unbalanced).
- **Tradeoff:** Lower loss, higher risk of overfitting small data.

CatBoost: Solving Target Leakage

The Issue: Target Encoding uses y_i to calculate feature $\hat{x}_i \implies$ Leakage.

Solution: Ordered Boosting

- **Concept:** Perform a random permutation of data.
- For example i , calculate target statistics using **only examples** $j < i$ in the permutation.
- **Analogy:** Simulating a time-series where you cannot see the future.

$$\hat{x}_i = \frac{\sum_{j < i} \mathbb{I}(x_j = c) \cdot y_j + a \cdot P}{\sum_{j < i} \mathbb{I}(x_j = c) + a}$$

Outcome: Allows using categorical features natively without manual encoding.

Week 1 Conclusion: The Hierarchy of Tabular Learning

- ① **Data Nature:** Tabular data is non-smooth and axis-aligned. Trees have the correct Inductive Bias.
- ② **Ensemble Theory:**
 - **Bagging:** Reduces Variance (Parallel).
 - **Boosting:** Reduces Bias (Sequential).
- ③ **Modern GBTs:** Mathematically rigorous (2nd order) and System optimized (Hessian-weighted sketches).

The Limit

All these models assume **Row Independence (I.I.D.)**.

Question: What if Row A (User) is related to Row B (Transaction)?

Next Week: We break the table. **Learning from Relational Data.**